

MANUAL DE USO DE GIT Y SUS COMANDOS

Para conectar la cuenta de git y que los commit queden bien guardados debemos iniciar con el *GLOBAL*:

-git config --list: muestra que información hay en el sistema

Git config --global user.email: le agregamos el correo

Git config --global user.name: le agregamos el usuario que queremos tener con el que se dejara en el historial de commit

```
C:\Users\ESTUDIANTE>git config --global user.name juandavid

C:\Users\ESTUDIANTE>git config --global user.email juan_da-mari@hotmail.com

C:\Users\ESTUDIANTE>git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=juandavid
user.email=juan_da-mari@hotmail.com
```

Git config --global --unset user.email: elimina el correo

```
ESTUDIANTE@CHIA68962 MINGW64 ~/IdeaProjects/prueba_git (main)
$ Git config --global --unset user.email juan_da-mari@hotmail.com
```

Git config --global --unset user.name: elimina el usuario:

```
C:\Users\ESTUDIANTE>Git config --global --unset user.name juandavid

C:\Users\ESTUDIANTE>git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
```

Git init "nombre": para inicializar el repositorio y crear las carpetas necesarias

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git> git init
Reinitialized existing Git repository in C:/Users/ESTUDIANTE/IdeaProjects/prueba_git/.git/
```

Git clone

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> git clone https://github.com/juandamari/practica_git.git
```

Git status

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git> git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  prueba_git/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git> git commit -m "comienzo"
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit (create/copy files and use "git add" to track)
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Git add .

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git> git add README.md
```

Git commit -m "comentario"

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git> git commit -m "first commit"
[master (root-commit) a85617a] first commit
 8 files changed, 83 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 .idea/.gitignore
 create mode 100644 .idea/misc.xml
 create mode 100644 .idea/modules.xml
 create mode 100644 .idea/vcs.xml
 create mode 100644 README.md
 create mode 100644 prueba_git.iml
 create mode 100644 src/Main.java
```

Git push origin rama: para instertar confirmaciones realizadas en la rama local en un repositorio

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> git push origin master
```

Git pull origin rama: descarga cambios del repositorio remoto y los integra en la rama local.

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> git pull origin rama
fatal: 'origin' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
```

Git branch -D "nombre de rama": irve para eliminar una rama local de Git de forma forzada.

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> Git branch -D repositorio
error: branch 'repositorio' not found
```

Git switch -c “nombre de rama nueva”: Sirve para crear una rama nueva y cambiarte automáticamente a ella.

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> Git switch -c nueva
Switched to a new branch 'nueva'
```

Git switch rama: sirve para cambiarte a una rama que ya existe.

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> git switch nueva
fatal: invalid reference: nueva
```

Git fetch --all: sirve para descargar desde los repositorios remotos la información más reciente

```
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> Git fetch --all
PS C:\Users\ESTUDIANTE\IdeaProjects\prueba_git\prueba_git> git status
On branch nueva

No commits yet
```

Git Branch -r: para ver que ramas remotas hay

```
ESTUDIANTE@CHIA68962 MINGW64 ~/IdeaProjects/prueba_git (main)
$ Git branch -r
origin/main
```

Git remote -v: para ver todas las ramas locales

```
ESTUDIANTE@CHIA68962 MINGW64 ~/IdeaProjects/prueba_git (main)
$ Git remote -v
origin https://github.com/juandamari/practica_git.git (fetch)
origin https://github.com/juandamari/practica_git.git (push)
```

Git log: muestra el código de registro de los comentarios

```
ESTUDIANTE@CHIA68962 MINGW64 ~/IdeaProjects/prueba_git (main)
$ Git log
commit a85617a93802efacf25fb86bd6ffb8d3983723b4 (HEAD -> main, origin/main)
Author: juandavid <juan_da-mari@hotmail.com>
Date: Thu Aug 20 20:39:58 2026 -0500

    first commit
```

Git reflog: sirve para ver el historial de movimientos que ha tenido head y tus ramas locales. Es especialmente útil para recuperar commits que creías haber perdido.

```
ESTUDIANTE@CHIA68962 MINGW64 ~/IdeaProjects/prueba_git (main)
$ git reflog
a85617a (HEAD -> main, origin/main) HEAD@{0}: Branch: renamed refs/heads/master to refs/heads/main
a85617a (HEAD -> main, origin/main) HEAD@{2}: commit (initial): first commit
```

Git log --oneline: sirve para ver el historial de commits de forma resumida, mostrando una sola línea por commit.

```
ESTUDIANTE@CHIA68962 MINGW64 ~/IdeaProjects/prueba_git (main)
$ Git log --oneline
a85617a (HEAD -> main, origin/main) first commit
```

Git merge: sirve para unir los cambios de una rama con otra rama.

```
ESTUDIANTE@CHIA68962 MINGW64 ~/IdeaProjects/prueba_git (main)  
$ Git merge  
Already up to date.
```

Git revert: sirve para deshacer los cambios de un commit, pero de una forma segura: crea un nuevo commit que revierte el commit anterior.

Git push origin - - delete "nombre de la rama": elimina la rama que no necesita